
Naresh Edagotti
@Statfusionai

Data Scientist's Hand Guide

A Comprehensive Guide for Beginners and Intermediate Learners

Python Essentials for Data Science

Basic Syntax & Variables

```
# Variable assignment
var = 10
name = "Data Science"
is_active = True
# Print statement
print(f"Value: {var}, Name: {name}")
```

Data Types & Type Conversion

- **Basic types:** int, float, str, bool, list, dict, set, tuple

```
# Type conversion
num_str = "42"
num_int = int(num_str) # String to integer
num_float = float(num_int) # Integer to float
```

Control Flow

```
# If-elif-else
if x > 0:
  print("Positive")
elif x == 0:
  print("Zero")
else:
  print("Negative")

# For loop
for i in range(5):
  print(i)

# While loop
while count < 10:
  count += 1
```

Functions & Lambda Expressions

```
# Function definition
def calculate_mean(numbers):
  return sum(numbers) / len(numbers)

# Lambda expression
square = lambda x: x ** 2
print(square(5)) # Output: 25
```

Data Structures

- **List:** Ordered, mutable collection
- **Dictionary:** Key-value pairs
- **Set:** Unordered collection of unique elements
- **Tuple:** Ordered, immutable collection

```
# List comprehension
squares = [x**2 for x in range(10)]
```

File Handling & Exception Handling

```
# File handling
with open('data.csv', 'r') as file:
  data = file.read()

# Exception handling
try:
  result = 10 / 0
except ZeroDivisionError:
  print("Cannot divide by zero")
```

Virtual Environments

```
# Create virtual environment
python -m venv myenv

# Activate virtual environment
# Windows: myenv\Scripts\activate
# Unix/MacOS: source myenv/bin/activate

# Install packages
pip install numpy pandas matplotlib
```

Working with Data Using Pandas & Polars

DataFrames and Series

```
# Pandas
import pandas as pd
df = pd.DataFrame({
  'A': [1, 2, 3],
  'B': ['a', 'b', 'c']
})
series = df['A']

# Polars
import polars as pl
df = pl.DataFrame({
  'A': [1, 2, 3],
  'B': ['a', 'b', 'c']
})
series = df.select('A')
```

Reading/Writing Data

Pandas

```
df = pd.read_csv('data.csv')
df.to_excel('output.xlsx')
df.to_json('data.json')
df.to_parquet('data.parquet')
```

Polars

```
df = pl.read_csv('data.csv')
df.write_excel('output.xlsx')
df.write_json('data.json')
df.write_parquet('data.parquet')
```

Filtering & Selecting Data

Pandas

```
df['column']
df.loc[:, 'column']
df[df['column'] > 5]
df.query('column > 5')
```

Polars

```
df.select('column')
df.filter(pl.col('column') >
5)
# No index, no .loc/.iloc
```

GroupBy & Aggregation

Pandas

```
df.groupby('category').agg({
  'value': 'mean'
})
df.groupby('category').sum()
```

Merging & Joining

Pandas

```
pd.concat([df1, df2])
pd.merge(df1, df2, on='key')
df1.join(df2, on='key')
```

Pivot Tables & Missing Data

```
# Pivot table
# Pandas
pivot_df = df.pivot_table(
  index='category',
  columns='subcategory',
  values='value',
  aggfunc='mean'
)

# Handling missing data
# Pandas
df.fillna(0)
df.dropna()

# Polars
df.fill_null(0)
df.drop_nulls()
```

Key Differences: Pandas vs Polars

- **Memory:** Polars uses Apache Arrow, Pandas uses Nu
- **Performance:** Polars has more parallel operations
- **Lazy evaluation:** Polars supports both eager and la
- **API:** Polars has expression-based API, more versatil

NumPy for Numerical Computation

Arrays: Creation, Indexing, Slicing

```
import numpy as np

# Array creation
arr = np.array([1, 2, 3, 4, 5])
zeros = np.zeros((3, 4))
ones = np.ones((2, 3))
random = np.random.rand(3, 3)

# Indexing and slicing
arr[0] # First element
arr[1:4] # Elements from index 1 to 3
arr[:3] # First 3 elements
arr[-1] # Last element
```

Array Operations & Broadcasting

```
# Basic operations
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = a + b # Element-wise addition
d = a * 2 # Scalar multiplication

# Broadcasting
A = np.ones((3, 3))
B = np.array([1, 2, 3])
C = A + B # B broadcasted to match A's shape
```

Useful Functions

- **np.where:** Conditional selection
- **np.argmax/argmin:** Index of max/min values
- **np.sum/mean/std:** Statistical operations

```
# Example of np.where
arr = np.array([1, 2, 3, 4, 5])
result = np.where(arr > 2, arr, 0)
# Output: [0, 0, 3, 4, 5]
```

Random Number Generation

```
# Setting seed for reproducibility
np.random.seed(42)

# Generate random numbers
rand_floats = np.random.rand(3, 3)
rand_ints = np.random.randint(0, 10, size=5)

# Random distributions
normal = np.random.normal(0, 1, 1000)
uniform = np.random.uniform(-1, 1, 100)

# Random sampling
choices = np.random.choice([1, 2, 3, 4, 5], size=3)
```

Linear Algebra

```
# Dot product
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
dot_product = np.dot(a, b)

# Matrix operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
matrix_mult = np.matmul(A, B)

# Matrix inverse and determinant
inv_A = np.linalg.inv(A)
det_A = np.linalg.det(A)

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
```

Key Advantages of NumPy

- **Performance:** Optimized C/Fortran backend
- **Memory efficiency:** Contiguous memory blocks
- **Vectorization:** Eliminates Python loops
- **Broadcasting:** Flexible array operations

Statistics for Data Science

Σ Measures of Central Tendency & Variability

- **Mean:** Average of all values
- **Median:** Middle value in sorted dataset
- **Mode:** Most frequently occurring value
- **Variance:** Average of squared deviations
- **Standard Deviation:** Square root of variance

```
import numpy as np
import pandas as pd

data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
mean = np.mean(data)
median = np.median(data)
variance = np.var(data)
std_dev = np.std(data)
```

Probability Basics

- **Probability:** Measure of likelihood (0 to 1)
- **Conditional Probability:** $P(A|B) = P(A \cap B)/P(B)$
- **Bayes' Theorem:** $P(A|B) = P(B|A) \times P(A)/P(B)$

```
# Simple probability calculation
def probability(event_outcomes, total_outcomes):
  return event_outcomes / total_outcomes

# Example: Probability of rolling a 3 on a die
prob_3 = probability(1, 6) # 0.1667
```

Distribution Types

Normal

Bell-shaped curve, symmetric around mean. Defined by μ (mean) and σ (std dev).

Binomial

Number of successes in n trials. Defined by n (trials) and p (success probability).

Poisson

Number of events in fixed interval. Defined by λ (average rate of occurrence).

Exponential

Time between events in Poisson process. Defined by λ (rate parameter).

Hypothesis Testing

- **Null Hypothesis (H_0):** Default assumption
- **Alternative Hypothesis (H_1):** Claim to be tested
- **p-value:** Probability of observing data if H_0 is true
- **Significance level (α):** Threshold for rejecting H_0

```
from scipy import stats

# One-sample t-test
sample = [1, 2, 3, 4, 5]
t_stat, p_value = stats.ttest_1samp(sample, 3)

# Chi-square test
observed = [[10, 20], [30, 40]]
chi2, p, dof, expected =
stats.chi2_contingency(observed)

# ANOVA
group1 = [1, 2, 3]
group2 = [4, 5, 6]
group3 = [7, 8, 9]
f_stat, p_value = stats.f_oneway(group1, group2,
group3)
```

Confidence Intervals

- **Definition:** Range of values likely to contain population parameter
- **Confidence level:** Probability that interval contains parameter (e.g., 95%)

```
import numpy as np
from scipy import stats

# 95% confidence interval for mean
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
mean = np.mean(data)
std_err = stats.sem(data)
ci = stats.t.interval(0.95, len(data)-1, loc=mean,
scale=std_err)
# Output: (3.33, 7.67)
```

Bayesian Statistics Basics

- **Prior Probability:** Initial belief about parameter
- **Likelihood:** Probability of observed data given parameter
- **Posterior Probability:** Updated belief after observing data

```
# Bayesian inference example
def bayesian_update(prior, likelihood, evidence):
  # Calculate posterior using Bayes' theorem
  posterior = (likelihood * prior) / evidence
  return posterior

# Medical test example
prior_disease = 0.01 # 1% prevalence
sensitivity = 0.95 # P(Test+|Disease)
specificity = 0.90 # P(Test-|No Disease)
# Calculate P(Disease|Test+)
```

Exploratory Data Analysis (EDA)

Descriptive Statistics

```
import pandas as pd
import numpy as np

# Basic statistics
df.describe()
df.info()
df.shape

# Specific statistics
df['column'].mean()
df['column'].median()
df['column'].std()
df['column'].skew()
df['column'].kurtosis()
```

Data Cleaning

- **Handling missing values:** Drop or impute
- **Data type conversion:** Ensure correct types
- **Removing duplicates:** Drop duplicate records

```
# Handling missing values
df.dropna() # Drop rows with missing values
df.fillna(0) # Fill missing with 0
df.fillna(df.mean()) # Fill with mean

# Type conversion
df['column'] = df['column'].astype('int')

# Removing duplicates
df.drop_duplicates()
```

Outlier Detection

- **Statistical methods:** Z-score, IQR
- **Visualization:** Box plots, scatter plots

```
# Z-score method
z_scores = (df['column'] - df['column'].mean()) /
df['column'].std()
outliers = df[np.abs(z_scores) > 3]

# IQR method
Q1 = df['column'].quantile(0.25)
Q3 = df['column'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['column'] < (Q1 - 1.5*IQR)) |
(df['column'] > (Q3 + 1.5*IQR))]
```

Correlation Analysis

- **Correlation matrix:** Relationships between variables
- **Heatmaps:** Visualize correlation matrix

```
# Correlation matrix
corr_matrix = df.corr()

# Finding highly correlated features
high_corr = corr_matrix[
  (corr_matrix > 0.7) | (corr_matrix < -0.7)
]

# Heatmap with seaborn
import seaborn as sns
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
```

Visualization Libraries

Matplotlib

Foundation for plotting in Python. Highly customizable but requires more code for complex plots.

Seaborn

Built on Matplotlib. Provides high-level interface for attractive statistical graphics.

Plotly

Creates interactive, publication-quality graphs. Great for web applications and dashboards.

Pandas Plotting

Built-in plotting functionality in Pandas. Convenient for quick visualizations directly from DataFrames.

Visualization Examples

```
# Matplotlib
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.scatter(df['x'], df['y'])
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.title('Scatter Plot')
plt.show()

# Seaborn
sns.histplot(data=df, x='column', kde=True)
sns.pairplot(df, hue='category')

# Plotly
import plotly.express as px
fig = px.scatter(df, x='x', y='y', color='category')
fig.show()
```

Data Preprocessing & Feature Engineering

Encoding Categorical Variables

- **One-Hot Encoding:** Creates binary columns for each category
- **Label Encoding:** Assigns unique integer to each category

```
from sklearn.preprocessing import OneHotEncoder,
LabelEncoder

# One-Hot Encoding
encoder = OneHotEncoder(sparse=False)
encoded_data =
encoder.fit_transform(df[['categorical_column']])

# Label Encoding
label_encoder = LabelEncoder()
df['encoded_column'] =
label_encoder.fit_transform(df['categorical_column'])
```

Feature Scaling

- **Standardization:** Transform to have mean=0, std=1
- **Normalization:** Scale to range [0,1] or [-1,1]

```
from sklearn.preprocessing import StandardScaler,
MinMaxScaler

# Standardization
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[['feature1',
'feature2']])

# Normalization
normalizer = MinMaxScaler()
normalized_data =
normalizer.fit_transform(df[['feature1',
'feature2']])
```

Binning & Discretization

- **Equal-width binning:** Divides range into equal intervals
- **Equal-frequency binning:** Each bin has same number of samples

```
import pandas as pd
import numpy as np

# Equal-width binning
bins = np.linspace(min(df['age']), max(df['age']), 6)
df['age_bin'] = pd.cut(df['age'], bins)

# Equal-frequency binning
df['age_quantile'] = pd.qcut(df['age'], 5, labels=
['Q1', 'Q2', 'Q3', 'Q4', 'Q5'])
```

Feature Selection Methods

- **Filter methods:** Statistical measures (correlation, chi-square)
- **Wrapper methods:** Use model performance to select features
- **Embedded methods:** Feature selection as part of model training

```
from sklearn.feature_selection import SelectKBest,
chi2, RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import Lasso

# Filter method
selector = SelectKBest(score_func=chi2, k=5)
X_new = selector.fit_transform(X, y)

# Wrapper method
rfe = RFE(estimator=RandomForestClassifier(),
n_features_to_select=5)
X_rfe = rfe.fit_transform(X, y)
```

Handling Imbalance

- **SMOTE:** Synthetic Minority Over-sampling Technique
- **Undersampling:** Reduce majority class samples
- **Class weights:** Adjust model's loss function

```
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import
RandomUnderSampler
from sklearn.utils import class_weight

# SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)

# Undersampling
undersampler = RandomUnderSampler(random_state=42)
X_res, y_res = undersampler.fit_resample(X, y)
```

Pipeline Building with sklearn

- **Pipeline:** Chain multiple transformers and estimator
- **ColumnTransformer:** Apply different transformers to different columns

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

# Define preprocessing for numeric and categorical
features
numeric_features = ['age', 'income']
numeric_transformer = Pipeline(steps=[
  ('imputer', SimpleImputer(strategy='median')),
  ('scaler', StandardScaler())
])

# Create preprocessing and modeling pipeline
preprocessor = ColumnTransformer(
  transformers=[
    ('num', numeric_transformer, numeric_features)
  ]
)

model = Pipeline(steps=[
  ('preprocessor', preprocessor),
  ('classifier', RandomForestClassifier())
])
```

Machine Learning (ML) Fundamentals

Learning Approaches

- **Supervised Learning:** Uses labeled data to train models
- **Unsupervised Learning:** Finds patterns in unlabeled data
- **Reinforcement Learning:** Learns through rewards/penalties

```
# Supervised learning example
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
```

Model Types

- **Regression:** Predicts continuous values
- **Classification:** Predicts discrete categories
- **Clustering:** Groups similar data points
- **Dimensionality Reduction:** Reduces feature space

Key Algorithms

Linear/Logistic Regression

Models relationship between variables using linear functions. Logistic for classification.

Decision Trees

Tree-like model of decisions and their possible consequences. Easy to interpret.

Random Forest

Ensemble of decision trees. Reduces overfitting and improves accuracy.

K-Nearest Neighbors

Classifies data points based on majority vote of k nearest neighbors.

Support Vector Machines

Finds hyperplane that best separates classes. Effective in high-dimensional spaces.

K-Means

Partitions data into k clusters by minimizing within-cluster variance.

Evaluation Metrics

- **Classification:** Accuracy, Precision, Recall, F1-Score
- **Regression:** MSE, RMSE, MAE, R^2
- **ROC-AUC:** Trade-off between true positive and false positive rates

```
from sklearn.metrics import accuracy_score,
precision_score, recall_score
from sklearn.metrics import mean_squared_error,
r2_score
from sklearn.metrics import roc_auc_score, roc_curve

# Classification metrics
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

# Regression metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

Cross-Validation & Hyperparameter Tuning

- **Cross-Validation:** Evaluates model performance on multiple data splits
- **Grid Search:** Exhaustive search over parameter space
- **Random Search:** Random sampling of parameter space

```
from sklearn.model_selection import cross_val_score,
GridSearchCV

# Cross-validation
scores = cross_val_score(model, X, y, cv=5)
print(f"Mean accuracy: {scores.mean():.2f}")

# Grid search
param_grid = {'n_estimators': [50, 100, 200],
              'max_depth': [None, 10, 20]}
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

Bias-Variance Tradeoff

- **Bias:** Error from incorrect assumptions in model
- **Variance:** Error from sensitivity to small fluctuations
- **Underfitting:** High bias, low variance (model too simple)
- **Overfitting:** Low bias, high variance (model too complex)
- **Goal:** Find balance between bias and variance

```
# Regularization to reduce overfitting
from sklearn.linear_model import Ridge, Lasso

# Ridge regression (L2 regularization)
ridge = Ridge(alpha=0.5)
ridge.fit(X_train, y_train)

# Lasso regression (L1 regularization)
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```

Deep Learning (DL) Basics

Neural Networks Architecture

- **Input Layer:** Receives initial data
- **Hidden Layers:** Process features through weighted connections
- **Output Layer:** Produces final prediction

```
# Simple neural network in PyTorch
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
  def __init__(self):
    super(SimpleNN, self).__init__()
    self.fc1 = nn.Linear(10, 5) # Input to hidden
    self.fc2 = nn.Linear(5, 1) # Hidden to output

  def forward(self, x):
    x = torch.relu(self.fc1(x))
    x = self.fc2(x)
    return x
```

Forward & Backward Propagation

- **Forward Propagation:** Data flows from input to output
- **Backward Propagation:** Updates weights based on error

```
# Forward propagation
model = SimpleNN()
inputs = torch.randn(1, 10)
outputs = model(inputs)

# Backward propagation
loss = nn.MSELoss()(outputs, torch.tensor([[1.0]]))
loss.backward() # Compute gradients
optimizer.step() # Update weights
```

Activation Functions

- **ReLU:** f(x) = max(0, x) - Most common for hidden layers
- **Sigmoid:** f(x) = 1/(1+e^(-x)) - Binary classification
- **Softmax:** Multi-class classification output layer

```
# Using activation functions in PyTorch
x = torch.tensor([-1.0, 0.0, 1.0, 2.0])

# ReLU
relu_output = torch.relu(x)

# Sigmoid
sigmoid_output = torch.sigmoid(x)

# Softmax
softmax_output = torch.softmax(x, dim=0)
```

Optimizers & Loss Functions

- **SGD:** Stochastic Gradient Descent - Basic optimizer
- **Adam:** Adaptive Moment Estimation - Most popular
- **MSE:** Mean Squared Error - Regression tasks
- **Cross-Entropy:** Classification tasks

```
# Setting up optimizer and loss function
import torch.optim as optim

# Optimizers
sgd_optimizer = optim.SGD(model.parameters(),
lr=0.01)
adam_optimizer = optim.Adam(model.parameters(),
lr=0.001)

# Loss functions
mse_loss = nn.MSELoss()
ce_loss = nn.CrossEntropyLoss()
bce_loss = nn.BCELoss() # Binary cross-entropy
```

Popular Deep Learning Frameworks

TensorFlow

Google's framework with Keras API. Production-ready with extensive deployment options.

PyTorch

Facebook's framework with Pythonic design. Popular in research and growing in production.

```
# Basic TensorFlow example
import tensorflow as tf

model = tf.keras.Sequential([
  tf.keras.layers.Dense(64, activation='relu'),
  tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Specialized Neural Networks

- **CNN:** Convolutional Neural Networks - Image processing
- **RNN:** Recurrent Neural Networks - Sequential data
- **LSTM:** Long Short-Term Memory - Addresses RNN limitations

```
# CNN example in PyTorch
class SimpleCNN(nn.Module):
  def __init__(self):
    super(SimpleCNN, self).__init__()
    self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
    self.pool = nn.MaxPool2d(2, 2)
    self.fc1 = nn.Linear(16 * 16 * 16, 10)

  def forward(self, x):
    x = self.pool(torch.relu(self.conv1(x)))
    x = x.view(-1, 16 * 16 * 16)
    x = self.fc1(x)
    return x
```

Projects and Real-world Applications

End-to-End ML Project




```
# Example end-to-end project structure
project/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   └── exploration.ipynb
├── src/
│   ├── preprocessing.py
│   ├── train.py
│   └── predict.py
├── models/
│   └── model.pkl
└── app.py
```

Time Series Forecasting

- **ARIMA:** Autoregressive Integrated Moving Average
- **Prophet:** Facebook's forecasting tool
- **LSTM Networks:** Deep learning for sequences

```
from statsmodels.tsa.arima.model import ARIMA
from fbprophet import Prophet

# ARIMA model
model = ARIMA(train_data, order=(5,1,0))
model_fit = model.fit()
forecast = model_fit.forecast(steps=12)

# Prophet model
df_prophet = df.rename(columns={'date': 'ds',
'value': 'y'})
model = Prophet()
model.fit(df_prophet)
future = model.make_future_dataframe(periods=12)
```

NLP Basics

- **Tokenization:** Splitting text into tokens
- **Vectorization:** Converting text to numerical vectors
- **Embeddings:** Word representations (Word2Vec, GloVe)

```
import nltk
from sklearn.feature_extraction.text import
TfidfVectorizer
from gensim.models import Word2Vec

# Tokenization
text = "Data science is amazing"
tokens = nltk.word_tokenize(text)

# TF-IDF Vectorization
corpus = ["Data science is amazing", "I love data"]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# Word2Vec
sentences = [['data', 'science'], ['i', 'love',
'data']]
model = Word2Vec(sentences, min_count=1)
```

Recommendation Systems

- **Collaborative Filtering:** User-item interactions
- **Content-Based:** Item features and user preferences
- **Hybrid:** Combines both approaches

```
from surprise import Dataset, KNNBasic
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.metrics.pairwise import
cosine_similarity

# Collaborative filtering with Surprise
data = Dataset.load_builtin('ml-100k')
trainset = data.build_full_trainset()
algo = KNNBasic()
algo.fit(trainset)

# Content-based filtering
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movie_overviews)
cosine_sim = cosine_similarity(tfidf_matrix,
tfidf_matrix)
```

Model Interpretability

- **SHAP:** SHapley Additive exPlanations
- **LIME:** Local Interpretable Model-agnostic Explanations

```
import shap
import lime
import lime.lime_tabular

# SHAP explanation
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)

# LIME explanation
explainer = lime.lime_tabular.LimeTabularExplainer(
  X_train.values, feature_names=X_train.columns)
exp = explainer.explain_instance(

  X_test.values[0], model.predict_proba,
num_features=5)
```

Model Deployment & MLOps Overview

Saving/Loading Models

- **Joblib:** Efficient for NumPy arrays, better for large models
- **Pickle:** Python's serialization module, more general

```
# Using joblib
from joblib import dump, load

# Save model
dump(model, 'model.joblib')

# Load model
model = load('model.joblib')

# Using pickle
import pickle

# Save model
with open('model.pkl', 'wb') as f:
  pickle.dump(model, f)

# Load model
with open('model.pkl', 'rb') as f:
  model = pickle.load(f)
```

Serving with Flask/FastAPI

Flask

```
from flask import Flask, request,
jsonify
import joblib

app = Flask(__name__)
model =
joblib.load('model.joblib')

@app.route('/predict', methods=
['POST'])
def predict():
  data = request.get_json()
  prediction =
model.predict([data['features']])
  return jsonify({'prediction':
prediction.tolist()})

if __name__ == '__main__':
  app.run(debug=True)
```

FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model =
joblib.load('model.joblib')

class InputData(BaseModel):
  features: list

@app.post('/predict')
def predict(data: InputData):
  prediction =
model.predict([data.features])
  return {'prediction':
prediction.tolist()}
```

Docker Basics

- **Dockerfile:** Text document with commands to assemble image
- **Image:** Read-only template with instructions for container
- **Container:** Runnable instance of an image

```
# Dockerfile example
FROM python:3.8-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--
host", "0.0.0.0"]

# Build and run commands
docker build -t ml-api .
docker run -p 8000:8000 ml-api
```

CI/CD for ML

- **Continuous Integration:** Automate testing and validation
- **Continuous Delivery:** Automate release process
- **Continuous Training:** Automate model retraining

```
# GitHub Actions example
(.github/workflows/ml.yml)
name: ML Pipeline

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-
python@v2
        with:
          python-version: 3.8
      - name: Install dependencies
        run: |
          pip install -r
requirements.txt
      - name: Train and evaluate
model
        run: python train.py
      - name: Deploy to production
        if: github.ref ==
'refs/heads/main'
        run: python deploy.py
```

Model Monitoring

- **Performance Monitoring:** Track accuracy, latency, throughput
- **Data Drift:** Detect changes in input data distribution
- **Concept Drift:** Detect changes in relationship between inputs and outputs

Data Science Best Practices

Reproducibility

- **Set random seeds:** Ensure consistent results
- **Virtual environments:** Isolate project dependencies
- **Notebook organization:** Clear structure and documentation

```
# Setting random seeds for reproducibility
import numpy as np
import random
import tensorflow as tf

np.random.seed(42)
random.seed(42)
tf.random.set_seed(42)

# Creating virtual environment
python -m venv my_project_env
source my_project_env/bin/activate # Linux/Mac
my_project_env\Scripts\activate # Windows
```

Documentation

- **README files:** Project overview and setup instructions
- **Docstrings:** Function and class documentation
- **Inline comments:** Explain complex logic

```
# Function with docstring
def preprocess_data(df, target_column):
  """Preprocess the input dataframe for model
training.

  Args:
    df (pd.DataFrame): Input dataframe
    target_column (str): Name of target column

  Returns:
    tuple: (X, y) features and target arrays
  """
  # Handle missing values
  df = df.fillna(df.mean())
  # Separate features and target
  X = df.drop(columns=[target_column])
  y = df[target_column]
  return X, y
```

Code Readability

- **PEP 8 compliance:** Python style guide
- **Descriptive names:** Clear variable and function names
- **Modular code:** Break down complex functions

```
# Before: Poor readability
def f(x, y):
  z = []
  for i in x:
    if i > y:
      z.append(i)
  return z

# After: Improved readability
def filter_values_above_threshold(values, threshold):
  """Filter values that are above the given
threshold."""
  filtered_values = [value for value in values if
value > threshold]
  return filtered_values
```

Version Control & Dataset/Model Versioning

Git Basics

Track code changes, collaborate with team members, and maintain project history. Use branches for feature development.

Dataset Versioning

Track changes to datasets with tools like DVC or Git LFS. Maintain data lineage and ensure reproducibility.

Model Versioning

Track model iterations, performance metrics, and hyperparameters. Use MLflow or similar tools.

Repository Organization

Use clear folder structure for data, notebooks, source code, models, and documentation.

```
# Basic Git workflow
git init
git add .
git commit -m "Initial commit with data
preprocessing"
git checkout -b feature/new-model
# Make changes...
git add .
git commit -m "Add new model architecture"
git checkout main
git merge feature/new-model

# DVC for data versioning
dvc init
dvc add data/raw/dataset.csv
git add data/.gitignore data/raw/dataset.csv.dvc
git commit -m "Add raw dataset and track with DVC"
```